What a Game Asks For: A Certified Generator Measures Where an LLM Harness Stops on ARC-AGI-3

Cody Lee Walker
Independent Researcher
cody@codylee.ca

2026-11-05 (ARC Prize 2026 Paper Track)

Code, games and certificates: github.com/codylwalker/arc3-rungs (MIT-0, CC0). The certified paths: s2ar.dev/play/arc3/. The repository's README maps each number below to its source.

Abstract

A leaderboard shows that a harness fails on ARC-AGI-3, but not what it fails at: each public game asks for every capability at once, is played once, and has no known optimum. This paper presents an instrument that answers by variation: a generator on the ARCEngine API whose every level carries an optimum certified by breadth-first search, and paired games that differ in one respect with every rule and optimum held fixed. We measure on it the open harness of our Kaggle entry, which runs a 4-bit open-weights model under the competition's compute limits, and report three things: on 41 small generated games that vary the dynamics it clears 97% of the levels, and three augmentations of its dynamics raise no level count; a pre-registered test that hides the goal (collect every key, or cover every plate) finds no cost it could detect, 1.08 times the cost per completed level, t-interval [0.95, 1.22], against a predicted 1.15 or more; and across two identical runs a game's token count repeats (Spearman 0.79) and its score less (0.56), a draw that one run per game cannot show. The miss leads us to hypothesise that the harness brings a prior over goals, and that its frontier is goals that pick out a subset or an order; the next pair set, with its predictions fixed now, tests that. Our selected submission scored 29.98 on the public half of the hidden games, the best of eight daily runs (mean 24.1).

1 Introduction

ARC-AGI-3 [3] uses interactive games to measure skill-acquisition efficiency: the skill a system gains, set against the priors and the experience it brings and the difficulty of generalising from them [4]. A game an agent has never seen tests how fast it learns, not what it knows, so the organisers built the hidden games to be "intentionally out-of-distribution relative to the public set" [3]. We use the definition's three terms. Difficulty is what a game asks a solver to learn, and we vary it one capability at a time. Experience is one play of an unseen game. Priors are where our results stop: Section 4.2 hypothesises that the harness brings a learned prior over goals, an order in which it tries goals before any evidence, beyond the Core Knowledge priors the organisers build to. Each game is a 64x64 grid in 16 colours with a handful of actions and no instructions, and an agent is scored on moves: a level scores the square of the ratio of the human baseline's move count to the agent's, capped at 115%. The baseline is the upper median of the move counts of the players, out of ten first-timers, who completed the level; later levels weigh more, and a game's score is capped at the weighted share of its levels completed [3].

The strongest open entries are LLM harnesses: an open-weights model in a Python sandbox, shown the grid as images, text and segmented objects [6, 8]. With frontier models, coding agents

that write and check an executable world model of each game reach high scores on the public games [7, 20, 22]. Under the competition's own limits, one GPU and nine hours for 110 games, the open harness our entry runs scored 29.98 on the public half of the hidden games in the submission we selected (Section 4.3).

When such a harness fails, what is it failing at? The leaderboard cannot say: each public game asks for every capability at once, so which games fail locates none; each is played once, so a score carries a draw no second run shows; and no public level has a known optimum. Two kinds of failure also look alike there, and we keep them apart throughout: a cost is a level the harness would clear with more time, and a wall is a level whose rule or goal more time would not bring. To see where a harness stops we need games whose structure we control, whose answers we know, and which we can run twice. This paper builds that instrument and reports what it shows about one strong harness.

Contributions:

1. **A ladder of what a game asks for,** ordered from the organisers' components [3], with an argument for where a solver that checks its models against its log can gain, scored against the results (Section 3.1). A public game asks for every rung at once, so which games a solver fails cannot isolate a rung (Section 3.2); we vary the rungs one at a time (Sections 3.3 and 4.2).
2. **A certified generator on the ARCEngine API,** released under MIT-0: seeded game plans over 13 mechanics, each level's optimum certified by search, and goals whose visibility on the HUD (the head-up display strip above the board) switches without a rule changing (Section 3.3).
3. **What it shows about one strong harness** (Sections 4.1 and 4.3): at small scale the dynamics of 13 mechanics do not stop it, and augmenting its dynamics gains nothing a run can show; across identical runs a game's token count repeats and its score less.
4. **A pre-registered test of the goal rung** (Section 4.2): paired games identical in every rule and optimum, the goal signalled in one and shown only by its consequence in the other. It finds no cost it could detect, and leaves as a hypothesis, with its test, that a solver's prior over goals does the work its evidence cannot.

2 Related work

ARC Prize papers. The first-place 2024 paper trained on a large synthetic set of generated programs and found that induction and transduction solve different tasks [12]; in 2025 the top three awards went to a tiny recursive network [11], self-improving evolutionary program synthesis [18] and compression without pretraining [13]; each builds a solver, where our generator measures one.

Solvers on ARC-AGI-3. Tufa Labs' Duck harness [6] runs a model in a Python REPL with image, ASCII and segmentation views of the grid and evicts old context; Franzen's Milestone-2 solution [8] builds on it with faster long-context serving, compute allocation across games and richer perception, and Affectify's slot priority [1] changes only its scheduler. Rodionov [19] finds that every harness variant improves with model capability and reasoning effort, gains that often exceed the differences between the variants. With frontier models the strongest solvers build executable world models: Rodionov's coding agent builds, checks and simplifies a Python world model of each game and plans in it, fully solving 15 of the 25 public games with GPT-5.5 [20]; Twin [22] has a coding agent build a test-time digital twin, check it against replayed observations and repair it at each mismatch, scoring 93.3% with a frontier model, and concludes: "Building a usable world model

is simpler than anticipated, whereas the harder problem is inferring the right goal." OPINE-World [7] keeps a "Bayesian, exploration-centric" space of hypothesised mechanics. Our harness's prompt asks for the same split between inferring a model and searching in it, advising an explicit search over the state space the model infers [8]; our augmentations (Section 3.4) test parts of that split under the competition's compute.

The public set as a test. Without a language model, a training-free graph explorer that ranks actions by visual salience placed third on the preview competition's private board [21]; our symbolic agent (Section 3.2) adds the induction of local rules by minimum description length, the principle CompressARC also uses [13]. One analysis reports every public game reachable through non-intelligent strategies, 18 of them in one step through a vulnerability confirmed in the library's local version, and concludes that only the private games are a genuine test [14]; under the ARC library's version 0.9.9 our own memoryless policies, random play without memory, reached a first level on 3 of the 25 public games within 300 actions (Section 3.3).

Controlled and generated evaluation. Procedurally generated levels are a standard way to measure generalisation in reinforcement learning [5]; bsuite isolates core capabilities in small controlled experiments [17]; XLand-MiniGrid samples each task as a ruleset: a goal and the rules that reach it [16]; and WorldTest queries whole environments to measure world-model learning [23]. For the static ARC tasks, RE-ARC writes an example generator for each training task [10], and ConceptARC groups tasks by concept [15]. Unlike these, each of our levels carries its certified optimum, so a solver's moves are scored against a known answer as ARC-AGI-3 scores moves, and our goal pairs vary one rung with every rule and optimum held fixed.

3 The instrument and the argument

3.1 The ladder, and what it predicts

The organisers name four components of what a game tests: exploration, modeling, goal-setting, and planning and execution [3]. We split modeling into perception and dynamics and add efficiency, which the score rewards, giving six rungs: *perception* (the objects, the agent, and what changed after an action), *dynamics* (the local rule of each object under each action), *goal* (what counts as done, which may show only as a consequence), *exploration* (the action that tells rival hypotheses apart), *planning* (a short path inside the induced model) and *efficiency* (few moves, since the score is squared against a human baseline).

A finite deterministic level has a value: the length of its shortest winning action sequence. A solver that has the rule and the goal can find that value by search, so what it must acquire is the rule and the goal. Evidence for a model of them counts only where it could have refuted the model, since evidence supports one hypothesis better than another if it is likelier under the first [9]; in a deterministic game, that happens only on a transition where a rival model predicts wrongly. If every model the solver entertains agrees on the log, a check against the log licenses no preference among them, however many transitions it matches.

Three consequences follow, and Table 1 scores each. A solver that has the rule and the goal plays near the optimum, so its cost is the learning before the search. A check of a world model against the log adds evidence only when the log holds discriminating transitions, which an exploration step has to create, and a planner adds value only once the goal is known. And a goal that shows only as a consequence has nothing to check against until the level completes: no evidence about the dynamics separates goal hypotheses that agree on every observed transition, so a solver strong on the dynamics should stall on the goal. Only the third was pre-registered, as the goal-rung predictions of Section 4.2; the first two are readings of the argument. In these terms the two

regimes of Section 1 are a cost that met the clock, which more time would clear, and a wall, where more time would not bring the rule or the goal.

Table 1. The argument of Section 3.1 scored against the results.

the argument says	in	result
given rule and goal, search nears the optimum	4.1	median level 1.12 times it; a fit, not a test
a log check gains only on discriminating transitions	4.1	probe never called; no gain (one run)
a goal shown only by its consequence stalls a solver	4.2	missed; revised to a prior over goals

3.2 Why a public failure cannot locate a rung

We built a symbolic agent rung by rung, as a second instrument: it tracks objects (its consistency is one minus its rate of ambiguous matches per frame transition), induces each object's local dynamics by minimum description length over object roles, explores, gates on a goal, and plans by breadth-first search inside the induced model. The LLM harness is far stronger, and the two stop at different rungs (Table 2). The agent's dynamics are local and its planning exact, but no version of it ever inferred a goal from play, so it stops at the goal rung by design; the harness guesses goals, clears the first level almost everywhere and loses its levels later. The public games the harness scores low on, outside the ten its notebook's self-test plays (the demonstration games), have consequence goals, state kept on the HUD, and long solutions. The two solvers' failures show no overlap beyond chance, a weak test, since a solver that gets past the first level of so few games can rule out only a large overlap. The point does not need it. Every public game asks for all the rungs at once, so where a solver fails on it is a fact about the solver and the game together, not about the game alone. That a game's difficulty depends on the solver is the premise of the paper that introduced the benchmark [4]; locating a rung takes varying it, on games whose answers we know.

Table 2. The two solvers on the 25 public games; the harness's figures are its author's logged run [8]. The agent's dynamics figure is the accuracy with which its induced rules predict each action's outcome. A game fails the agent when it never gets past level 1; the last two rows test whether the two solvers' failures overlap.

on the 25 public games	the symbolic agent	the LLM harness [8]
perception	tracking consistent at 0.997	given: image, text, objects
dynamics	0.80 or better on 11 of 25 games	writes world-model functions
goal	never inferred; 6 of 25 past level 1	guessed; past level 1 on 24 of 25
planning	level 1 of ls20: 17 moves, human 22	by hand, or its own search
mean score	0.3	46.5
demonstration games; other fifteen		78.7; 25.1
harness score, agent passed / failed		45.3 on 6 / 46.9 on 19 ($P = 0.53$)
games both fail (harness under 50)		10 vs 10.64 expected ($P = 0.86$)

3.3 The generator, its certificates and the goal pairs

The generator composes a seeded game plan: one to four mechanics from thirteen (hazards, ice, keys and doors, pushable blocks, one-way doors, teleporters, wrap-around, snake trails, two-agent switching, state flips and three click families), a palette with a distinct colour per role, a per-game shuffle of which action moves which way, and six levels that grow, the first holding the first mechanic alone. The HUD carries the keys held and the budget bar, in the public games' style. Each level is then certified:

1. Search breadth-first over the game's exported state from the level's start, each action an edge, until a state completes the level. The search is capped at 50,000 states and 120 seconds a level, so the generator emits only levels small enough to search exhaustively.

2. Keep the level only if its optimum is at least 3 moves (2 on a click game's first level); otherwise redraw it.
3. Write the certificate: the optimal path, a human-style baseline (1.5 times the optimum, rounded up) and a budget (3 times the optimum).
4. Replay every emitted file against its certified paths before it is kept, and rerun the certifier on it to confirm the optimum.

The certifier certifies any goal whose variables are in the exported state; a test that hides the keys from its snapshot confirms that a missing goal variable cannot pass silently (a level whose optimum is 6 is reported unsolvable). The baseline is ours, not measured from people, and a game whose levels are all played within it scores the ceiling of 100. The set (Table 3) was built from the public API alone. Memoryless policies, random play without memory, clear a larger share of its levels than of the public ones: our levels are easier, which Section 4.1 bears out.

Table 3. The generated set and the goal-rung pairs, with two checks: memoryless play at 300 actions a game, and the file-access scan of the five full runs of Table 4 and the goal-rung run.

the generated set	
games, levels	41, 246
mechanics a plan draws	one to four of 13
levels a game	six, growing from 7x7 to 18x14 cells
median certified optimum	11 moves
levels memoryless policies clear	20 of 246; 3 of the 183 public levels
goal-rung pairs	10 pairs, five keys and five plates: 20 games, 120 levels
code blocks scanned for file access	11,401 in five runs, 4,369 in the goal-rung run; none names an input path

Two goal kinds have no goal tile. A level completes when every key is collected, or when every plate is covered by a block. Each such plan is emitted twice. In the signalled variant the HUD's last row shows one pip per item, filled as it is done. In the consequence variant nothing marks the goal until the level completes, except that in the keys pairs both variants keep the key slots the keydoor mechanic draws on the HUD, a running count of keys held. Both variants come from one plan and one certification, and the same paths replay on both, so they differ only in the HUD's goal row (Figure 1).

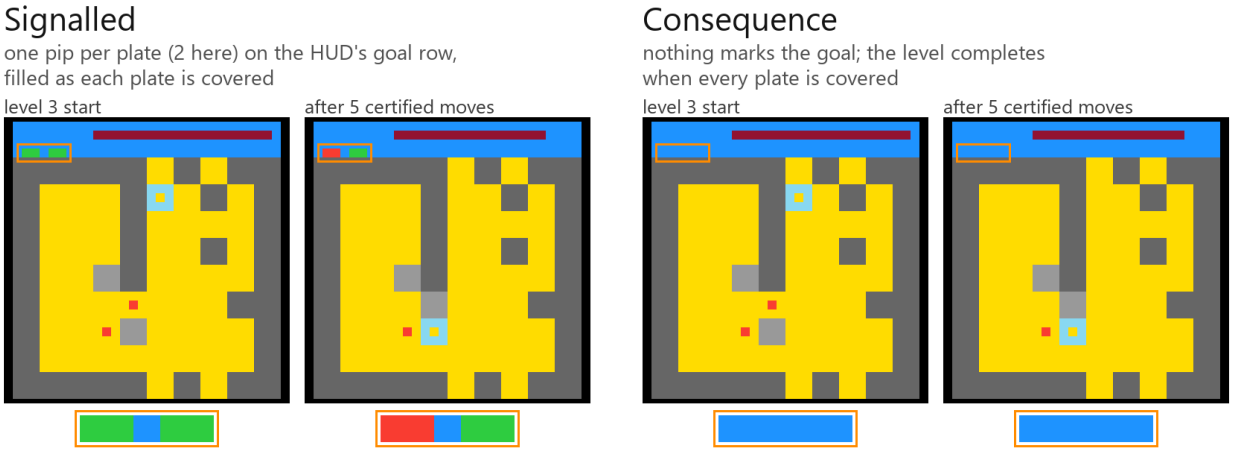


Figure 1: One goal-rung pair: a plates level with the goal signalled on the HUD (left: one pip per plate, filled as each is covered) and shown only by its consequence (right), at the start and after five of its thirteen certified moves. Same rules, optimum and moves; the outlined band, enlarged under each frame, is the only place the frames differ.

3.4 The harness, its three changes and the augmentations

Our leaderboard submission is Franzen's Milestone-2 notebook [8], built on Tufa Labs' Duck harness [6]. In outline it works as follows. All 110 game sessions start at once on the GPU's model server, and a scheduler keeps ten games active, freezing the rest and admitting first the games whose progress looks most promising. Each turn the model sees the board as an image scaled 10x, as ASCII text and as segmented objects, with difference images and any animation frames, and writes Python in a REPL that holds the game's actions, UNDO among them. A guard stops a snippet's further actions once an action leaves the board unchanged. The functions the model defines persist across turns and levels, and old context is trimmed in blocks.

We changed three things. The scheduler's priority is the D' formula Affectify published [1], which we reimplemented. The model loads from a smaller quantisation, which loads faster. And a keep-alive holds the scorecard open while the model loads, against the gateway's 15-minute idle close, which had zeroed our first two submissions (Table 11). The model is Qwen3.8-Flash-Next, a 125B-parameter mixture of experts with 6B active, quantised to 4-bit weights, on the competition's single RTX PRO 6000 GPU. The keep-alive only lets a run score at all, and this paper claims no score gain from the other two changes: the single draws we have could not show one, since our runs with and without the model-checking toolkit described below, and with and without the slot priority, differ by less than the spread of eleven teams' runs of the unchanged notebook (Table 11). The contribution is the measurement.

The venue asks that the linked submission demonstrate the paper's approach. It demonstrates the harness the paper measures, under the paper's three changes, with the tools we built wired in and switched off; the instrument, a generator and a certifier, is not a solver and cannot sit on a leaderboard. In the generated-games runs each game gets 40 minutes, ten games at a time behind the model server, on the competition's free compute and never submitted. A pilot before these runs gave each game about a quarter of that model time, with all 41 games running at once; Table 4 lists it for scale. Three augmentations are tested against the base, the submitted harness unchanged:

- **A model-checking toolkit** in the prompt and the namespace: check a world model against the log, check a goal hypothesis, plan in the model, execute a plan until surprised, and choose the most informative probe.
- **An unasked certificate,** of the model's world model rather than of a level: after each code snippet, any world-model function the model defined is replayed over the level's last 60 logged transitions, and one line reporting the match goes into the snippet's output.
- **A text diff with HUD meters:** Franzen's frame diff switched on, noting objects more than three cells away from what the action moved that appeared, vanished, recoloured or resized, and each budget bar's rate.

3.5 How the runs are read

Predictions were written before each run and are scored verbatim. For the first run, the base harness's, we fixed the scoring rules after seeing its first table, which breaks that practice, so the release's script prints every alternative reading beside each verdict; the goal-rung predictions and their rules were fixed before the set was generated. Each run is compared with a run of the base notebook, game by game, and with a placebo: two runs of the identical notebook, whose difference is one draw of what chance alone produces, a scale for an arm's difference rather than a test of it. A pattern found by looking gets the exact null of the search that found it. Intervals are game bootstraps unless marked as t-intervals; both omit run-level variance, such as throughput and scheduling, which the placebo samples once, so an interval that barely excludes zero is weaker than it looks [2]. Cost is measured as generated characters per completed level, paired by game and

level, because on a clock-bound run total tokens are throughput times elapsed time, and tokens per level solved only restates the level count; whole-game comparisons use token counts. Cost per completed level leaves out the levels an arm fails, so it is read beside the level count. A game-run is one game's play in one run, and in the goal-rung run, which plays each game five times, each play is a game-pass and each level within it a level-pass. Before any result is read, the model's code is scanned for file access (Table 3).

4 Results

Table 4 lists the runs on the 41 generated games; a game-run is unfinished when the clock ends it with levels left. The three sections that follow take the abstract's findings in turn: the dynamics rung, the goal rung, and the draw that one run per game carries.

Table 4. The runs on the 41 generated games (246 levels), each game played once a run; scores are against our human-style baseline.

run	levels cleared	games won	mean score	unfinished
base, the submitted harness	239	38	89.6	3
base, repeated	240	38	90.7	3
toolkit, first version	235	37	89.2	4
toolkit, second version, certificate	225	34	82.5	7
diff and meters	235	38	90.3	3
pilot, a quarter of the model time	114	16	40.8	25

4.1 The dynamics rung is not where it stops

On the generated games the base harness cleared almost every level and won almost every game, and its repeat did the same (Table 5). No level was solved in fewer moves than its certified optimum, a check on the certificates rather than on the harness, and most solved levels were played at or near it; the scores sit high partly because the baseline is ours, and fall when every baseline is set to the optimum instead of 1.5 times it. In nearly every game the model's own code defines a search, as its prompt advises: a match on the code finds a queue, the word bfs or a routine named for solving or searching, though not that the search chose the moves. Its generation is heaviest on the first level and falls by a factor of five by the sixth (Figure 2), which we read as the mechanic learned once, since the levels grow while the cost falls, though the per-game action shuffle, learned on the first level, and the functions the harness keeps across levels would give the same curve. Our predictions before the runs placed the difficulty in the mechanics or in a level's length, and none held in full (Table 5). The pilot, with about a quarter of the model time, cleared under half the levels (Table 4), so time is the likelier lever, though it also ran all 41 games at once.

Table 5. The base harness in two identical runs; a blank cell was not measured separately for the repeat.

the base harness	first run	repeat
levels cleared of 246	239 (97%)	240
games won of 41	38	38
games scored 100 of 41	22	23
median time to win a game	18 minutes	
solved levels played at the optimum	42%	40%
median ratio of a level's moves to its optimum	1.12	1.18
mean score with every baseline at the optimum	70.2	69.6
games whose code defines a search	40 of 41	41 of 41
median generated characters on levels 1, 2 and 6	54,000; 23,000; 11,000 (both runs, Figure 2)	
predictions made before the run	3 missed, 2 half-hit, 1 untestable	

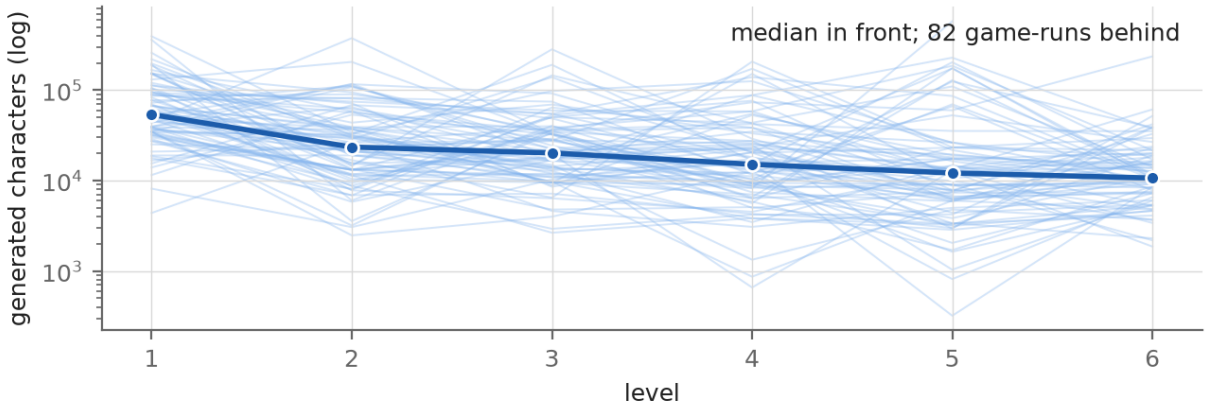


Figure 2: Generated characters per level on the 41 generated games: one line per game-run over the two base runs, the median in front, log scale. The cost is highest on the first level and falls about five times by level 6, which we read as the rule learned once and reused.

If the dynamics at this scale are already in hand, an augmentation of the dynamics has little left to add, and that is what the three arms found (Tables 6 and 7). They were run against the base in three runs, the toolkit in two versions and the certificate only with the second. The toolkit's first version masks the budget bar by its position in the frame's edge band, where nearly every public game with a bar draws it; our generator, like one public game, draws its HUD inside the frame, so every plan the model executed stopped at its first or second move on the bar, and the model seldom called a tool at all. The second version masks by behaviour instead (a colour whose count moves one way on at least 90% of its changes, on at least 30% of a level's actions) and adds the unasked certificate. Some plans now ran to completion, the certificate's line was right where checked and twice led the model to repair its world model, and the arm cleared fewer levels, running several games into the clock.

The diff and meters tied on levels, at a cost within the difference between two identical runs. On the public demonstration games, where every run is clock-bound, they cut cost per completed level against earlier tools-off demonstration runs, which ran other set-ups and have no placebo, so the saving may be noise; there they also spent more of their generation on the unfinished level than the submitted harness does.

Table 6. Against the mean of the two base runs: levels fewer a game, and cost per completed level paired by game and level, positive when costlier; game-bootstrap intervals. The placebo row sets the base's repeat against its first run; the demonstration-games row compares with earlier tools-off runs.

arm	levels of 246	levels fewer a game	cost vs base
toolkit, first version	235		
toolkit, second version, certificate	225	0.35 [0.02, 0.76]	
diff and meters	235	0.11 [-0.17, 0.46]	+4.8% [-3.2, 13.9]
base, repeated: the placebo	240		+4.9% [-8.2, 20.3]
diff and meters, demonstration games			-6.1% [-12.1, -0.2]

Table 7. Inside the augmentation arms, with the public games' bars that the first version's mask assumed.

inside the arms	
public games drawing their bar in the edge band	23 of 24
plans the model executed that ran to completion	0 of 5 (first version); 5 of 14 (second)
games in which the model called a tool, first version	8 of 41
games the certificate arm ran into the clock	7
certificate lines	in 8 games, right where checked; 2 repairs
generation on the unfinished level, demo games	diff and meters 20 to 21%; submitted harness 13%

No augmentation raised the level count, and none could have shown much of one: the base runs left a handful of levels to add, a gain smaller than the width of every interval here. The certificate arm's loss, in one run that changed the mask and added the certificate together, may be the time its plans' stops and the certificate's replays cost. Section 3.1 suggests a further, untested reason, that the model never called the probe that would make a model check discriminating; whether a check helps where the harness's model is wrong can only be tested on games it fails.

4.2 The goal rung: a pre-registered miss and a revision

The goal rung asks what counts as done. A public game never says: a level completes when a condition the player was not told is met, and the player learns it from what happens. The goal-rung set isolates that one thing. Each of its ten plans has a goal with no goal tile, collect every key or cover every plate, and is emitted as two games from one plan and one certification (Section 3.3, Figure 1). In the signalled variant the HUD's last row shows one pip per item, filled as it is done; in the consequence variant that row is blank, and nothing marks the goal until the level completes. Every rule, level and per-level optimum is shared (Table 3), and the same certified paths replay on both, so whatever the harness pays in the consequence variant beyond the signalled is the price of finding the goal. Section 3.1's argument predicted a stall there, since no transition before completion can refute a wrong goal hypothesis. The harness plays each game five times, five game-passes in one run. The predictions and their reading rules were fixed before the set was generated; each compares the consequence variant with the signalled, and P-G1 and P-G2 also require the interval to exclude no effect (Table 8, above the divider). The control held and the four predictions missed: across both variants the harness cleared nearly every level-pass, and on level 1, where a hidden goal must first be found, the cost ratio was below one, against a small excess on the later levels (Table 8, below the divider, and Figure 3; both exploratory). A first run of the set was void, its model server crashing minutes into play, and was repeated unchanged.

Table 8. The goal-rung test. Above the divider row, the predictions fixed before the set was generated, with their results: P-G1 and P-G2 also require their interval to exclude no effect, P-G2's interval is the pre-registered game-bootstrap, and the level-passes row is a count. Below it, the readings made after the fact, which Figure 3 draws: the cost per completed level, consequence over signalled, as t-intervals over pairs and as bootstrap intervals by level group.

prediction	rule	result
P-G1, levels	at least 0.5 fewer levels a game-pass	0.0 [0, 0], miss
P-G2, cost	at least 1.15 times per level	1.08 [0.97, 1.20], miss
P-G3, level 1	the gap largest on level 1	0.0 against 0.0 levels, miss
P-G4, the clock	more unfinished, sign test $p < 0.10$	1 against 1, miss
P-G5, control	signalled clears at least 85%	99.3%, held
all level-passes	cleared, both variants	596 of 600
after the fact, exploratory (Figure 3)		
cost, all ten pairs	t-interval	1.08 [0.95, 1.22]
cost, keys pairs	t-interval	1.03 [0.87, 1.22]
cost, plates pairs	t-interval	1.13 [0.86, 1.48]
cost, level 1	bootstrap	0.96 [0.76, 1.16]
cost, levels 2 to 6	bootstrap	1.10 [0.97, 1.24]

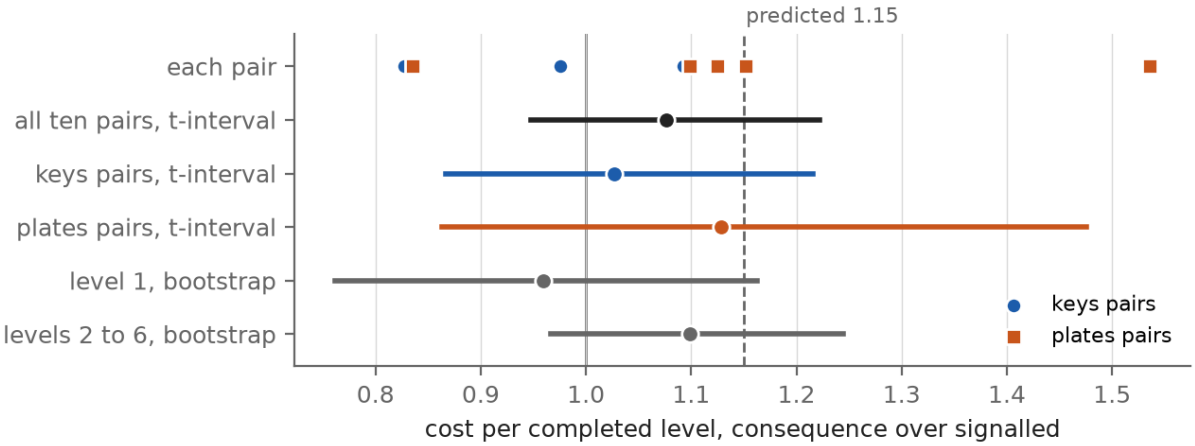


Figure 3: Cost per completed level in the consequence variant over the signalled: one dot per pair, then the t-interval over all ten pairs and over each goal kind, and the bootstrap intervals for level 1 and for levels 2 to 6; Table 8 lists the values. The solid line is no cost; the dashed line is the predicted 1.15.

Nothing shows that the harness reads the pips, and the set held no pair built to stall; the test also had less power than its rules suggest. With every level well within the clock, the level readings had little room, and P-G4, with about one unfinished game-pass a variant, could not have succeeded. P-G2 asked for a point estimate at or above the predicted cost, which a true cost of that size would meet only half the time, and neither interval excludes the predicted 1.15; each rules out only a ratio above about 1.2. In the keys pairs both variants show the keys held, so the five plates pairs are the cleaner test; they show a larger excess on a wider interval. A cost below the test's reach, a signal the harness never reads, or a solver's prior over goals would each give this result.

Section 3.1 predicted a stall on these simple consequence goals, collect everything or cover everything, and P-G3 placed it on level 1 above all, because in the consequence variant nothing marks the goal until the level completes (the keys pairs' count of keys held aside). The level-1 ratio, below one though its interval reaches the predicted cost, gives that mechanism no support, since a goal found by evidence would cost most where it is first found. A failed prediction refutes its assumptions together and does not say which to change; the choice is a judgement, which a later test can bear out but not show to be the only one. The one we change, after the result and as a hypothesis, is the assumption that a solver needs evidence to find a goal. A solver also brings a prior, an order in which it tries goals before any evidence (acquired, not the Core Knowledge priors every game assumes [3]). Call a goal board-suggested when it acts on every object of one kind the board shows: a board of keys, or of blocks and plates, suggests the goal these levels have, and every goal in the set is of that kind. The symbolic agent, with no prior over goals, never infers a goal; the harness guesses, and in its author's run clears the first level of nearly every public game (Table 2). Twin's finding that inferring the right goal is the harder problem [22] fits the revision: the goals that cost are the ones the board does not suggest, those that pick out a subset or an order. The next pair set tests it with the same five readings and with pairs built to stall: covering only the plates of one colour, or collecting the keys in a fixed order. If the pairs built to stall cost no more than these did, the revision fails; a cost above these pairs' but below the thresholds of Section 5 leaves the test inconclusive.

4.3 One run per game is a draw, on our games and on the hidden set

Two runs of one notebook agree on each game's token count more than on its score against our baseline, by a difference whose interval barely excludes zero (Table 9 and Figure 4); their level counts differ by a fraction of a level a game on average. The token count is not a proxy for a game's size: with each game's total certified optimum partialled out, it agrees as well. Nor is the score's weaker agreement its ceiling's doing, though more than half the games score 100 in each run: with every baseline at the optimum, where no game reaches 100, the scores agree less still. On the games both runs scored 100, where the score separates nothing, token counts still agree: a game has a token cost for this harness that repeats on games it wins, which the score cannot see; on the games both runs scored under 100, neither agrees detectably. A game's token count in one run also weakly predicts whether it ends unfinished in another, and the two identical runs share none of their unfinished games, which chance alone gives most of the time, so two runs cannot show whether failures repeat.

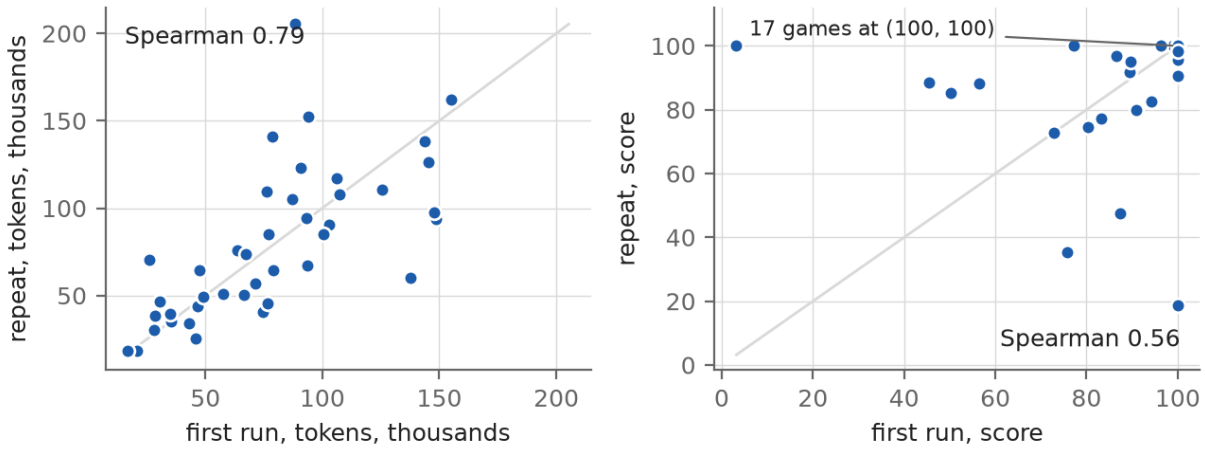


Figure 4: Two runs of the identical notebook on the 41 generated games: each game's token count (left) and score (right) in the first run against the repeat, with the diagonal. Token counts repeat at Spearman 0.79; the score agrees at 0.56.

Table 9. Two identical runs: agreement per game (Spearman unless stated; 95% game-bootstrap intervals where given). The AUC rows predict whether a game ends unfinished from its token count in another run; the three runs are the two base runs and the toolkit's first version.

two identical runs	games	agreement
levels a game, mean difference	all 41	+0.02 [-0.24, 0.32]
token counts	all 41	0.79 [0.63, 0.89]
scores	all 41	0.56 [0.29, 0.80]
token-count agreement minus score agreement	all 41	0.23 [0.00, 0.48]
token counts, game size partialled out	all 41	0.79 [0.61, 0.89]
token counts	the 17 both runs scored 100	0.82 [0.54, 0.94]
token counts	the 13 both runs scored under 100	0.46 [-0.25, 0.85]
scores	the 13 both runs scored under 100	0.10
scores, every baseline at the optimum	all 41	0.48
token count predicts an unfinished game, AUC	the two orderings	0.65, 0.74
the same over six ordered pairs of three runs	median	0.77

Five runs can say more (Table 10). Failures concentrate on some games, though the test rejects only a uniform spread and the runs are not exchangeable, the certificate arm contributing a third of

them; two links noticed by looking, between the failures and the switch and click-move mechanics, got the exact null of the search that found them and did not survive the runs made after they were noticed.

Table 10. Unfinished game-runs across the five full runs of Table 4 (the pilot aside).

unfinished game-runs	
across five runs	20, in 13 distinct games; 16.6 expected if uniform (exact P = 0.014)
from the certificate arm	7
shared by the two identical runs	0, which chance gives four times in five
most runs any one game failed, of five	3
switch games, two runs after noticing	2 of 22 game-runs vs 5 of 60 for the rest (one-sided p = 0.61)
click-move games, the run after	1 of 3 vs 2 of 38 for the rest (p = 0.21)

The leaderboard shows the same draw at full scale. Identical code scored differently on each of eight daily runs of the hidden games, spread almost as widely as eleven teams' runs of the unchanged notebook (Table 11), and each logged run used all its time. The competition's game server allows one run per game, so a best-of-runs rule never applies, and the final rank takes the other half of the selected run. We selected the best public draw: its run-wide luck, such as throughput and scheduling, carries to the same run's private half; its per-game luck does not.

Table 11. The submissions on the public half of the hidden games, in order. The toolkit-on run used the earlier quantisation and the toolkit-off run the smaller one; the eight days add the slot priority to the toolkit-off code.

the hidden games, public half	score
the first two submissions, before the keep-alive	0.00
toolkit on	20.25
toolkit off	21.01
slot priority added: the same code on eight days	26.55, 21.88, 25.23, 29.98, 25.43, 21.05, 21.09, 21.58
their mean	24.1
the selected submission	29.98; public rank 1,008 of 5,070 at the close
eleven teams' runs of the unchanged notebook	18.0 to 29.1

5 Conclusion

At the scale of our generated games, a strong open harness has the dynamics rung: it writes the search its prompt advises, with its median level close to the optimum in both runs, and its difficulty there is cost. In the terms of Section 3.1, on our games a failure is a cost that met the clock: failures concentrate on some games, but none failed in more than three of the five runs. On public games that the harness clears in none of the few runs we have we call a failure a wall, and we extend the name to the hidden set on the organisers' design; we never varied the time given on a public game, and the runs are few, so that is an assumption, not a measurement. A game's token count weakly predicts the first kind of failure; nothing we measured predicts the second. The public games it fails differ from ours in what our first set held fixed: the goal rung, with goals that show only as consequences, and scale. Our paired test isolated the first and found no cost it could detect for goals that act on every object of a kind. The revision of Section 4.2 places a wall where a solver's prior does not reach and its exploration cannot separate the rivals; on that hypothesis our pairs' goals sit inside the prior, the hidden set was built out of distribution in its mechanics [3], and whether its goals lie outside the prior is untested.

For whoever builds the next harness, the measurement says four things. A score from one run per game carries a draw (Tables 9 and 11), so a change of a few levels on a few games is inside the noise until a second run says otherwise. When a harness that already searches nearly clears every game, no augmentation of its dynamics can show a gain. Goals that act on every object of a kind

cost this harness nothing measurable, and we hypothesise that the goals to build for, and to test on, are the ones the board does not suggest. And a detector that spots a wall early can recover at most what the harness spends there: the share of its generation the submitted harness spends on the level it never clears (Table 7), less if some of that is cost.

We measured one harness with one model, on small single-screen levels, with goals of two simple kinds, and one run per arm cannot fix the size of an effect of a few levels. The human-style baseline is ours, and it gives most generated games the ceiling score. The certifier's search is capped, so at scale a certificate can hold a path, an upper bound, but not yet the optimum.

On progress toward the benchmark's goal, this paper moves no score. The next pair set's predictions are fixed now: hiding a goal that picks out a subset or an order costs at least half a level a game-pass, or at least 1.15 times as much per completed level. The generator and its certificates are released so that the frontier can be found where the program can be read: the smallest certified change to a game that takes a harness from clearing a level to not clearing it.

6 Reproduction, licence and authorship

The repository, github.com/codylwalker/arc3-rungs, holds the generator, the certificates, the symbolic agent, the harness changes and the analysis scripts, with a README that maps each section's numbers to its script. Code is MIT-0 and certificates CC0; the anchor lines of our patches keep the open harness's Apache-2.0 notice. The Kaggle notebooks, the run outputs and the public games' trajectories are not redistributed. AI coding and writing tools were used throughout, at the author's direction and under the author's review; the experiments' design and every reading decision are the author's.

References

[1] Affectify. ARC-AGI-3 Milestone 2 Solution with a new slot priority (D'). *Kaggle notebook, kaggle.com/code/shiiin9/affectify-arc-31-54-in-a-single-sub*, 2026.

[2] R. Agarwal, M. Schwarzer, P. S. Castro, et al. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *NeurIPS, arXiv:2108.13264*, 2021.

[3] ARC Prize Foundation. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. *arXiv preprint arXiv:2603.24621*, 2026.

[4] F. Chollet. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[5] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman. Leveraging Procedural Generation to Benchmark Reinforcement Learning. *ICML, arXiv:1912.01588*, 2020.

[6] J. Cottaar and Tufa Labs. The Duck: ARC-AGI-3 Inference Harness, Winning Solution to ARC-AGI-3 Milestone 1. `https://github.com/Tufalabs/duck-harness`, 2026.

[7] D. Courtis, W. Li, and S. Sanner. OPINE-World: Programmatic World Modeling with Ontology-error-Prioritized Interactive Exploration for ARC-AGI-3. *arXiv preprint arXiv:2607.01531*, 2026.

[8] D. Franzen. ARC-AGI-3 Milestone 2: Building on the Duck Harness. `https://github.com/da-fr/arc-agi-3-solution`, 2026. Kaggle notebook, kaggle.com/code/dfranzen/arc-agi-3-milestone-2-solution.

[9] I. Hacking. Logic of Statistical Inference. *Cambridge University Press, 1965; reissue, p. 53*, 2016.

[10] M. Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. *arXiv preprint arXiv:2404.07353*, 2024.

[11] A. Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. *arXiv preprint arXiv:2510.04871*, 2025.

[12] W.-D. Li, K. Hu, C. Larsen, et al. Combining Induction and Transduction for Abstract Reasoning. *arXiv preprint arXiv:2411.02272*, 2024.

[13] I. Liao and A. Gu. ARC-AGI Without Pretraining. *arXiv preprint arXiv:2512.06104*, 2025.

[14] K. H. Liew. Explore Before You Solve: The Speed-Depth Trade-off in Epistemic Agents for ARC-AGI-3. *arXiv preprint arXiv:2605.25931*, 2026.

[15] A. Moskvichev, V. V. Odouard, and M. Mitchell. The ConceptARC Benchmark: Evaluating Understanding and Generalization in the ARC Domain. *TMLR, arXiv:2305.07141*, 2023.

[16] A. Nikulin, V. Kurenkov, I. Zisman, et al. XLand-MiniGrid: Scalable Meta-Reinforcement Learning Environments in JAX. *NeurIPS Datasets and Benchmarks, arXiv:2312.12044*, 2024.
[17] I. Osband, Y. Doron, M. Hessel, et al. Behaviour Suite for Reinforcement Learning. *ICLR, arXiv:1908.03568*, 2020.
[18] J. Pourcel, C. Colas, and P.-Y. Oudeyer. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI. *ICML, arXiv:2507.14172*, 2025.
[19] S. Rodionov. Do Coding Agents Need Executable World Models, Simplification, and Verification to Solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026.
[20] S. Rodionov. Executable World Models for ARC-AGI-3 in the Era of Coding Agents. *arXiv preprint arXiv:2605.05138*, 2026.
[21] E. Rudakov, J. Shock, and B. U. Cowley. Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks. *arXiv preprint arXiv:2512.24156*, 2025.
[22] A. Skoutnev, K. Acharya, G. Longhitano, M. Udell, K. Ellis, and I. Drori. Twin: Playing an Unknown Game with a Test-Time Digital Twin. *arXiv preprint arXiv:2608.14490*, 2026.
[23] A. Warrier, D. Nguyen, M. Naim, et al. Benchmarking World-Model Learning with Environment-Level Queries. *ICML, arXiv:2510.19788*, 2026.